Sound Sentry

A Sound-Triggered Security Camera System Powered by Edge AI on Raspberry Pi

Project Report

23CSE472 - Artificial Intelligence and Robotics

Amrita Vishwa Vidyapeetham
School of Computing, Coimbatore
Department of Computer Science and Engineering

Project Team
Amal Godwin J (CB.SC.U4CSE23407)
Haridev P (CB.SC.U4CSE23417)
Keshav RamPrakash (CB.SC.U4CSE23425)
Prajan Karthik (CB.SC.U4CSE23439)
Sidharth S Nair (CB.SC.U4CSE23443)
Manu Santhosh (CB.SC.U4CSE23729)

Mentor
Dr. A. Baskar

Contents

1 Introduction and Problem Definition **3**
- 1.1 Background and Overview . . . 3
- 1.2 Problem Statement . . . 3
- 1.3 Need for the Proposed Solution . . . 3
- 1.4 Objectives . . . 4
- 1.5 Scope and Application . . . 4
- 1.6 Existing System and Its Limitations . . . 5

2 Proposed System and Design **5**
- 2.1 Proposed Solution . . . 5
- 2.2 System Requirements . . . 6
- 2.3 System Architecture . . . 6
- 2.4 Block Diagram . . . 7
- 2.5 System Workflow . . . 8
- 2.6 Functional and Non-Functional Requirements . . . 8

3 AI and Robotics Methodology **9**
- 3.1 AI/ML Technique and Algorithm . . . 9
- 3.2 Dataset / Input Data . . . 9
- 3.3 Data Preprocessing and Model Development . . . 10
- 3.4 Robotics Components . . . 10
- 3.5 AI-Robotics Integration . . . 11
- 3.6 Algorithm and Flowchart . . . 12

4 Implementation, Testing and Results **13**
- 4.1 Hardware Implementation . . . 13
- 4.2 Software Implementation . . . 14
- 4.3 Working Model / Prototype . . . 15
- 4.4 Testing and Validation . . . 15
- 4.5 Performance Evaluation . . . 16
- 4.6 Results and Discussion . . . 16
- 4.7 Screenshots / Prototype Photographs . . . 18

5 Conclusion and Future Scope **18**
- 5.1 Summary of the Work . . . 18
- 5.2 Achievement of Objectives . . . 19
- 5.3 Limitations . . . 19
- 5.4 Future Enhancements . . . 20
- 5.5 Real-World Deployment Possibilities . . . 20
- 5.6 Learning Outcomes . . . 20
- 5.7 References . . . 21
- 5.8 Appendix . . . 22

1 Introduction and Problem Definition

1.1 Background and Overview

Security surveillance has long relied on continuous video recording. Conventional security cameras either operate as passive recording devices, requiring humans to sift through hours of footage or use motion detection that floods the system with false alarms from wind, wildlife and lighting changes. These limitations are increased in outdoor and perimeter scenarios where visual cues are obstructed by darkness, fog or physical barriers, but sounds such as breaking glass, door knocking or creaking, and footsteps in restricted areas still travel through the environment.

Recent advances in on-device deep learning have made it possible to deploy audio classification models directly on to the edge device such as Raspberry Pi [10]. By combining real-time audio intelligence with motorised Pan-Tilt-Zoom (PTZ) camera control, it is possible to build a system that listens for threats and autonomously looks in the direction of the sound source, without any cloud connectivity, internet dependency, or privacy-compromising data uploads.

Sound Sentry continuously captures stereo audio from two INMP441 I2S MEMS microphones [11], classifies a 1-second window every 0.5 seconds using a quantised YAMNet-based [1] neural network running on-device, estimates the direction of arrival (DOA) via Generalised Cross-Correlation with Phase Transform (GCC-PHAT) [5], and commands a TP-Link TL-SC4171G [8] PTZ IP camera to pan towards the suspicious sound. Model inference takes approximately 7-10 ms on the Raspberry Pi, and the design target for the time from a sound to the start of camera motion is under 1 second.

1.2 Problem Statement

Design and implement an edge-AI security system that accomplishes the following:

1. Continuously monitors ambient audio using a two-microphone array connected to a Raspberry Pi.
2. Classifies each audio window in real time as suspicious (footsteps, glass breaking, door knocking, door creaking) or non-suspicious (wind, rain, thunderstorm, engine, car horn, dog, crickets) using a quantised deep learning model running entirely on-device.
3. Estimates the direction of the suspicious sound source using time-difference-of-arrival (TDOA) techniques with sub-sample lag resolution.
4. Autonomously steers a pan-tilt IP camera to face the estimated direction of the suspicious sound.

1.3 Need for the Proposed Solution

Traditional security systems have many limitations that create a compelling need for an alternative approach. Visual-only motion detection generates excessive false alarms from environmental factors. Fixed cameras have a limited field of view, often missing events occurring outside their coverage area. Cloud-dependent systems require reliable internet connectivity, introduce processing latency, incur ongoing subscription costs, and raise significant data privacy and sovereignty concerns.

Audio-based threat detection offers inherent advantages over visual approaches. Sound propagates through darkness, around corners, through fog, and past physical obstacles that would completely block a camera's line of sight. Breaking glass, door knocking or creaking, and footsteps in a restricted area produce distinctive acoustic signatures that a trained neural network can recognise, even when these events occur outside the camera's current field of view.

The proposed system addresses these gaps by combining edge-based audio classification with robotic camera control. By processing all data on-device, the system removes the dependency

on cloud services and keeps audio on the premises. By steering the camera towards the sound source, it extends the effective coverage area from a fixed cone to a full 180-degree arc.

Limitation of Existing Systems	**How Sound Sentry Addresses It**
Visual-only motion detection produces excessive false alarms	Audio classification targets specific threat-related sound categories
Fixed cameras have limited field of view	PTZ motor autonomously steers to the direction of the sound source
Cloud-dependent systems require internet, have latency, raise privacy concerns	All inference runs on-device; no data leaves the premises
Manual monitoring requires constant human attention	Fully autonomous pipeline from detection to camera orientation
Standard cameras are blind in darkness or fog	Audio propagates through darkness, around corners, through fog

Table 1: Comparison with Existing Limitations

1.4 Objectives

1. Train a binary audio classifier using transfer learning on YAMNet with a custom dense classification head to discriminate between suspicious and normal environmental sounds.
2. Deploy the model at the edge as a quantised TensorFlow Lite [6] file of approximately 3.7 MB on a Raspberry Pi, with inference latency of at most 10 ms per 1-second audio window.
3. Implement GCC-PHAT direction of arrival estimation using a two-microphone linear array, with sub-sample lag resolution.
4. Develop a PTZ camera controller with dead-reckoned heading tracking, empirically calibrated motor speeds, automatic home-position re-synchronisation and error recovery.
5. Integrate all components into a real-time pipeline that processes overlapping audio windows every 0.5 seconds, with decision stages that limit unnecessary camera activations.

1.5 Scope and Application

In scope:

- Binary classification (suspicious vs. normal) of environmental sounds using a quantised deep learning model.
- Horizontal (pan-only) direction of arrival estimation in the angular range of -90 degrees to +90 degrees.
- HTTP-based PTZ control of a TP-Link TL-SC4171G IP camera via its CGI API.
- Deployment on Raspberry Pi 5 with 64-bit Raspberry Pi OS.
- Three operational modes: live microphone with real camera, dry-run mode, and WAV file replay mode.

Potential applications:

- Perimeter security for homes, offices and warehouses.
- Surveillance in low-visibility conditions such as night-time and fog.
- Restricted-area monitoring for server rooms, storage facilities and construction sites.
- Integration into broader smart-home or building-management systems.

1.6 Existing System and Its Limitations

The main existing surveillance approaches each have limitations that Sound Sentry seeks to address.

Existing Approach	Key Limitation
Passive CCTV recording	Requires manual review; no real-time alerting; cannot direct camera focus
PIR / motion-detection cameras	High false-positive rate from animals, wind and shadows; cannot respond to sounds
Cloud-based audio surveillance	Requires internet; introduces latency; raises data privacy and ongoing cost concerns
Multi-camera arrays	Expensive hardware; complex installation and maintenance; still rely on visual triggers
Human security guards	Costly; subject to fatigue, limited attention span and variable response time

Table 2: Existing Approaches and Their Limitations

2 Proposed System and Design

2.1 Proposed Solution

Sound Sentry is a complete edge-AI surveillance solution designed to run autonomously on a Raspberry Pi without any cloud connectivity. The system consists of five tightly integrated modules, each responsible for a distinct aspect of the sensing-to-actuation pipeline.

1. **Audio Capture Module:** Two INMP441 I2S MEMS microphones connected to the Raspberry Pi GPIO header via a breadboard, spaced 25 cm apart, capturing stereo audio at 48 kHz. The audio is accumulated in a thread-safe ring buffer with lock-protected access.
2. **AI Classification Module:** A quantised TFLite model combining the YAMNet feature extractor with a custom dense classification head. The model accepts 1-second windows of 16 kHz mono audio and outputs a single probability P(suspicious) via a sigmoid activation.
3. **Direction of Arrival Module:** GCC-PHAT (Generalised Cross-Correlation with Phase Transform) cross-correlation on the left and right audio channels, providing sub-sample lag resolution and azimuthal angle estimation using the arcsin formula.
4. **Camera Control Module:** An HTTP-based PTZ controller running on a background daemon thread. It communicates with the camera via CGI endpoints, supports both continuous and preset move modes, and implements dead-reckoned heading tracking with periodic absolute re-synchronisation.

5. **Orchestrator (Main Loop):** Ties all modules together, processing overlapping 1-second windows every 0.5 seconds through a multi-stage decision pipeline with loudness gate, camera-busy guard, classification threshold, and an optional consecutive-hit guard.

2.2 System Requirements

Hardware Requirements

Component	Specification
Computer	Raspberry Pi 5 (64-bit Raspberry Pi OS, kernel 6.x)
Microphones	2 x INMP441 I2S MEMS microphone modules (omnidirectional, 24-bit)
Camera	TP-Link TL-SC4171G Wireless Pan/Tilt IP Camera
Network	Ethernet cable connecting camera to Pi on the same subnet
Power	5V USB-C for Raspberry Pi; 12V adapter for camera

Software Requirements

Software	Details
Operating System	64bit - Raspberry Pi OS
Python	3.8 or higher
NumPy [12]	Array operations, signal processing, ring buffer management
SciPy [13]	resample_poly for anti-aliased audio resampling (48 kHz to 16 kHz)
sounddevice [9]	Real-time audio capture via PortAudio backend
requests	HTTP communication with the PTZ camera CGI API
TFLite Runtime [6, 7]	ai-edge-litert (preferred) / tflite_runtime/tensorflow
PortAudio	System library for low-level audio I/O (portaudio19-dev)

2.3 System Architecture

The system follows a modular architecture. Each module is implemented as a separate Python file with well-defined interfaces. The orchestrator (main.py) imports and connects all modules, managing the data flow from audio capture through classification and DOA estimation to camera actuation.

The architecture is designed around several key principles:

Design Decision	Rationale
Edge-only inference	No cloud latency; works offline; audio is not uploaded
TFLite quantisation	∼3.7 MB model, ∼7-10 ms inference on Pi
RMS loudness gate	Prevents the model from misfiring on silent/quiet audio
Camera busy/cooldown window	Prevents camera motor noise from triggering false detections
GCC-PHAT over direct cross-correlation	O(N log N) complexity, better suited to reverberant rooms
Continuous PTZ with move=repeat	One smooth pan instead of jerky 1.5-second chunk approach
Dead-reckoning + home re-sync	Limits accumulated heading error without encoder feedback
Background worker thread for camera	Audio stream is not blocked by HTTP requests

2.4 Block Diagram

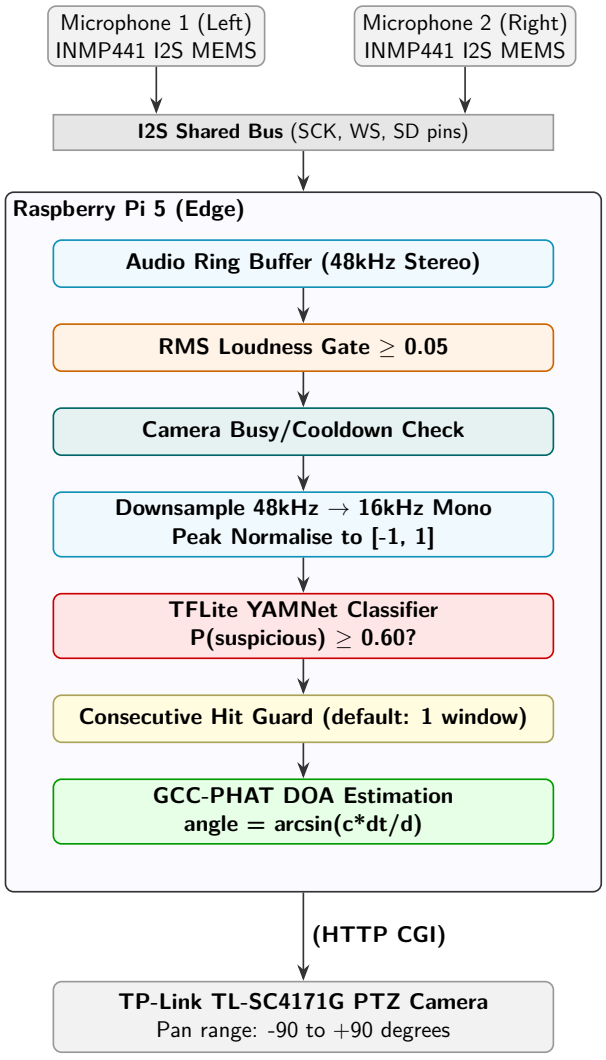


Figure 1: Sound Sentry System Architecture Block Diagram

2.5 System Workflow

The processing pipeline runs every 0.5 seconds on the most recent 1 second of stereo audio. The overlapping windows reduce the chance of a short sound being split across two windows. The workflow consists of six sequential stages, most of which can end the processing of a window early.

- **Step 1 - Audio Capture:** A `sounddevice.InputStream` callback accumulates audio blocks (100 ms each at 4800 samples/block) into a ring buffer. Every 0.5 seconds, the latest 1-second window (48000 samples $\times$ 2 channels) is copied under a threading lock for processing.
- **Step 2 - Loudness Gate:** If max(RMS_left, RMS_right) < 0.05, the window is silently discarded. This is required because the model was observed to return high suspicious scores on silent audio, so such windows must never reach it.
- **Step 3 - Camera-Busy Guard:** If the camera is currently executing a move or is within its 2-second post-move cooldown period, the window is skipped. This prevents the camera's own motor noise from being captured by the microphones and triggering another detection. Sounds that occur during this period are not analysed.
- **Step 4 - AI Classification:** The 48 kHz stereo window is downsampled to 16 kHz mono using `scipy.signal.resample_poly` with anti-aliasing, then peak-normalised to the range [-1, 1]. The processed audio is fed to the quantised TFLite model, which outputs P(suspicious). If P(suspicious) < 0.60, the window is classified as a normal sound.
- **Step 5 - Consecutive-Hit Guard:** If CONSECUTIVE_HITS > 1, that many consecutive windows must exceed the threshold before the system acts. The default value is 1, so the guard is inactive unless enabled with `--hits`.
- **Step 6 - Direction of Arrival and Camera Move:** GCC-PHAT cross-correlation between the left and right channels over the loudest 16384 samples of the detection window estimates the time delay. The delay is converted to an azimuthal angle via angle $= \arcsin(c \cdot dt/d)$. The camera controller receives the target angle, computes a move plan, and executes it on the background worker thread. Moves smaller than 5 degrees are skipped; targets within 7.5 degrees of 0 degrees use the absolute home preset for heading re-synchronisation.

2.6 Functional and Non-Functional Requirements

Functional Requirements

ID	Requirement
FR-01	Continuously capture stereo audio at 48 kHz from two I2S MEMS microphones
FR-02	Classify each 1-second audio window as suspicious or normal using the TFLite model
FR-03	Estimate the direction of arrival of suspicious sounds using GCC-PHAT
FR-04	Command the PTZ camera to pan towards the estimated direction
FR-05	Provide a loudness gate to suppress false positives on quiet audio
FR-06	Provide a camera-busy guard to prevent self-triggering from motor noise
FR-07	Support live microphone, WAV file replay, and dry-run modes
FR-08	Automatically return the camera to home position on exit and after idle
FR-09	Log all events with timestamped, colour-coded terminal output
FR-10	Track and display session statistics at exit

Non-Functional Requirements

ID	Requirement
NFR-01	Model inference latency ≤ 10 ms per 1-second window on Raspberry Pi
NFR-02	End-to-end response time (sound to camera motion start) < 1 second
NFR-03	Model file size ≤ 4 MB for edge deployment
NFR-04	Zero cloud dependency: all processing runs on-device
NFR-05	Graceful shutdown with camera return to home on Ctrl+C or a handled error
NFR-06	Lock-protected audio buffer, with camera commands issued from a single worker thread

3 AI and Robotics Methodology

3.1 AI/ML Technique and Algorithm

The system uses Binary Audio Classification via Transfer Learning. The core AI technique is transfer learning on Google's YAMNet [1], a deep convolutional neural network pre-trained on the AudioSet [3] dataset comprising 521 audio event classes and over 2 million labelled 10-second clips from YouTube. YAMNet uses a MobileNet V1 backbone [2], which was specifically designed for efficient inference on mobile and embedded devices.

Transfer learning suits this application because YAMNet's MobileNet backbone has already learned rich, general-purpose audio representations from its training on 521 diverse sound classes. By freezing the YAMNet weights and training only a small dense classification head, suspicious/normal discrimination can be learned with limited training data (a subset of ESC-50) and limited compute resources (a Kaggle notebook).

The choice of binary classification (suspicious vs. normal) rather than multi-class classification was deliberate. The operational question the system must answer is simply: 'Is this sound suspicious?' Binary classification simplifies the decision pipeline, reduces false-positive management complexity, and provides a single tunable threshold (SUSPICIOUS_THRESHOLD = 0.60) for sensitivity adjustment.

3.2 Dataset / Input Data

The model was trained on a curated subset of the ESC-50 [4] (Environmental Sound Classification) dataset, published by Karol J. Piczak at the 2015 ACM Multimedia conference. ESC-50 contains 2000 audio clips across 50 environmental sound classes, with 40 clips per class, each 5 seconds long, recorded at 44.1 kHz in WAV format. The 11 classes used here contain 440 clips: 160 suspicious and 280 background.

Category	Label	Classes Used from ESC-50
Suspicious	1	Footsteps, Glass breaking, Door wood knock, Door wood creaks
Background/Normal	0	Wind, Rain, Thunderstorm, Engine, Car horn, Dog, Crickets

Table 3: Class Mapping

The suspicious classes were chosen to represent sounds commonly associated with intrusion attempts or security-relevant events: glass breaking (window entry), door knocking and creaking (possible forced entry), and footsteps (unauthorized presence). The background classes were

chosen to represent common ambient sounds that should not trigger camera movement: weather (wind, rain, thunderstorm), traffic (engine, car horn), and wildlife (dog, crickets).

3.3 Data Preprocessing and Model Development

Data Preprocessing Steps

1. **Chunking:** The highest-energy 1-second segment is extracted from each 5-second ESC-50 clip. This ensures the model trains on the most acoustically informative portion of each recording, rather than on trailing silence or low-energy segments.

2. **Resampling:** All audio is resampled from 44.1 kHz to 16 kHz to match YAMNet's expected input sample rate. This resampling is performed with anti-aliasing to prevent spectral folding artefacts.

3. **Peak Normalisation:** Each clip is scaled to the range [-1, 1] by dividing by the absolute peak amplitude. This normalisation is critical because YAMNet's internal Mel-spectrogram computation expects input in this range, and the model produces unreliable outputs on unnormalised audio.

4. **Stratified Split:** The dataset is split 80% for training and 20% for validation, with stratification to preserve the class balance ratio across both subsets.

Model Development Pipeline

Training Environment: Kaggle notebook (sound-model-ai-robotics.ipynb)

1. **Base Model Loading:** YAMNet is loaded from TensorFlow Hub with all weights frozen (non-trainable). This preserves the rich audio representations learned from AudioSet.

2. **Head Architecture:** A custom classification head is attached: Global Mean Pooling (to aggregate frame-level embeddings into a single vector) followed by Dense (256, ReLU), Dropout(0.5), and Dense(1, Sigmoid).

3. **Training:** The model is trained using binary cross-entropy loss with the Adam optimiser. Only the dense head weights are updated; YAMNet's weights remain frozen throughout.

4. **End-to-End Export:** The complete model, including YAMNet's internal preprocessing (Mel-spectrogram extraction) and the classification head, is exported as a single TFLite file. This end-to-end design means the deployed model accepts raw audio samples directly.

5. **Quantisation:** Dynamic-range quantisation (`tf.lite.Optimize.DEFAULT`) reduces the model size from approximately 15 MB (full TensorFlow SavedModel) to approximately 3.7 MB (quantised TFLite). Inference time improves from approximately 50 ms to 7-10 ms on the Raspberry Pi. The model's input and output remain float32, so the calling code needs no integer conversion.

3.4 Robotics Components

Sensors

The system uses two INMP441 I2S MEMS microphone modules as its primary sensors. These are digital microphones that communicate via the I2S (Inter-IC Sound) protocol, providing 24-bit audio at sample rates up to 48 kHz. The omnidirectional polar pattern ensures sensitivity to sounds arriving from any direction, which is essential for the direction-of-arrival estimation.

Pin	Mic 1 (Left Channel)	Mic 2 (Right Channel)
VDD	Pi Pin 1 (3.3V)	Pi Pin 17 (3.3V)
GND	Pi Pin 6 (GND)	Pi Pin 9 (GND)
L/R	Tied to GND → left channel	Tied to VDD → right channel
SCK	Shared → Pi Pin 12 (Bit clock)	Shared → Pi Pin 12
WS	Shared → Pi Pin 35 (Word select)	Shared → Pi Pin 35
SD	Shared → Pi Pin 38 (Serial data)	Shared → Pi Pin 38

Actuators

The TP-Link TL-SC4171G Wireless Pan/Tilt IP Camera serves as the system's actuator. It provides motorised pan (horizontal rotation) controlled via an HTTP CGI API at the endpoint `/cgi-bin/operator/ptzset`. The camera supports both continuous motion commands (where the motor turns until explicitly stopped) and preset position commands (where the motor moves to a saved absolute position).

CGI Endpoint	Purpose
move=right&move=repeat	Start continuous right pan (motor turns until stop)
move=left&move=repeat	Start continuous left pan (motor turns until stop)
move=stop	Stop motor immediately
gotoserverpresetno=N	Move to saved preset position N (absolute)

Controller

The Raspberry Pi 5 serves as the central controller, running the complete software stack. Its ARM Cortex-A76 processor provides sufficient compute for real-time TFLite inference, FFT-based cross-correlation, and HTTP communication. The Pi's 40-pin GPIO header provides the I2S bus interface for the MEMS microphones, while its Ethernet port connects to the IP camera.

3.5 AI-Robotics Integration

The integration between the AI classification system and the robotic camera controller follows a classic sense-decide-act paradigm, adapted for real-time audio processing, with decision stages that limit unnecessary camera activations.

- **Sense:** Two MEMS microphones capture stereo audio continuously. The audio stream is buffered in a NumPy ring buffer with thread-safe access via a `threading.Lock`. Every 100 ms, a new block of 4800 samples is appended; every 500 ms, the latest 48000-sample (1-second) window is copied for processing.
- **Decide:** The multi-stage decision pipeline processes each window through the loudness gate, camera-busy guard, TFLite classifier, and consecutive-hit guard. Only when all guards pass and P(suspicious) ≥ 0.60 does the system proceed to DOA estimation.
- **Act:** The GCC-PHAT module computes the sound direction from the stereo audio. The camera controller translates this angle into a move plan (target angle, direction, duration, preset if applicable) and executes it on a background daemon thread. The background thread ensures that the 500 ms processing cadence is maintained even during long camera moves.

3.6 Algorithm and Flowchart

Main Decision Algorithm (executed every 0.5-second cycle):

```
FUNCTION process(window, current_time):
    rms_left <- RMS(window[:, 0])
    rms_right <- RMS(window[:, 1])
    IF max(rms_left, rms_right) < RMS_GATE THEN
        consecutive_hits <- 0
        RETURN 'quiet ignored'

    IF camera is moving OR current_time < busy_until THEN
        RETURN 'camera busy ignored'

    mono <- downsample_to_16kHz(mean(channels))
    mono <- peak_normalise(mono)
    P_suspicious <- TFLite_model.predict(mono)

    IF P_suspicious < SUSPICIOUS_THRESHOLD THEN
        consecutive_hits <- 0
        RETURN 'normal sound'

    consecutive_hits <- consecutive_hits + 1
    IF consecutive_hits < CONSECUTIVE_HITS THEN
        RETURN 'accumulating hits'

    consecutive_hits <- 0
    segment <- loudest_segment(window, DOA_SEGMENT)
    lag <- GCC_PHAT(segment[:, 0], segment[:, 1])
    angle <- arcsin(clip(lag/fs * c/d, -1, 1))

    IF |angle - camera.heading| < MIN_TURN_DEGREES THEN
        RETURN 'already pointing there'

    camera.move_to(angle)
    RETURN 'SUSPICIOUS camera moving to angle'
```

GCC-PHAT Direction of Arrival Algorithm:

```
FUNCTION gcc_phat_lag(left, right, max_lag):
    nfft <- next_power_of_2(len(left) + len(right))
    spec <- FFT(left) * conj(FFT(right))
    spec <- spec / (abs(spec) + epsilon)  // phase whitening
    cc <- IFFT(spec)
    cc <- extract_lags(cc, -max_lag..+max_lag)
    k <- argmax(cc)
    shift <- parabolic_interpolation(cc, k) // sub-sample
    RETURN k - max_lag + shift
```

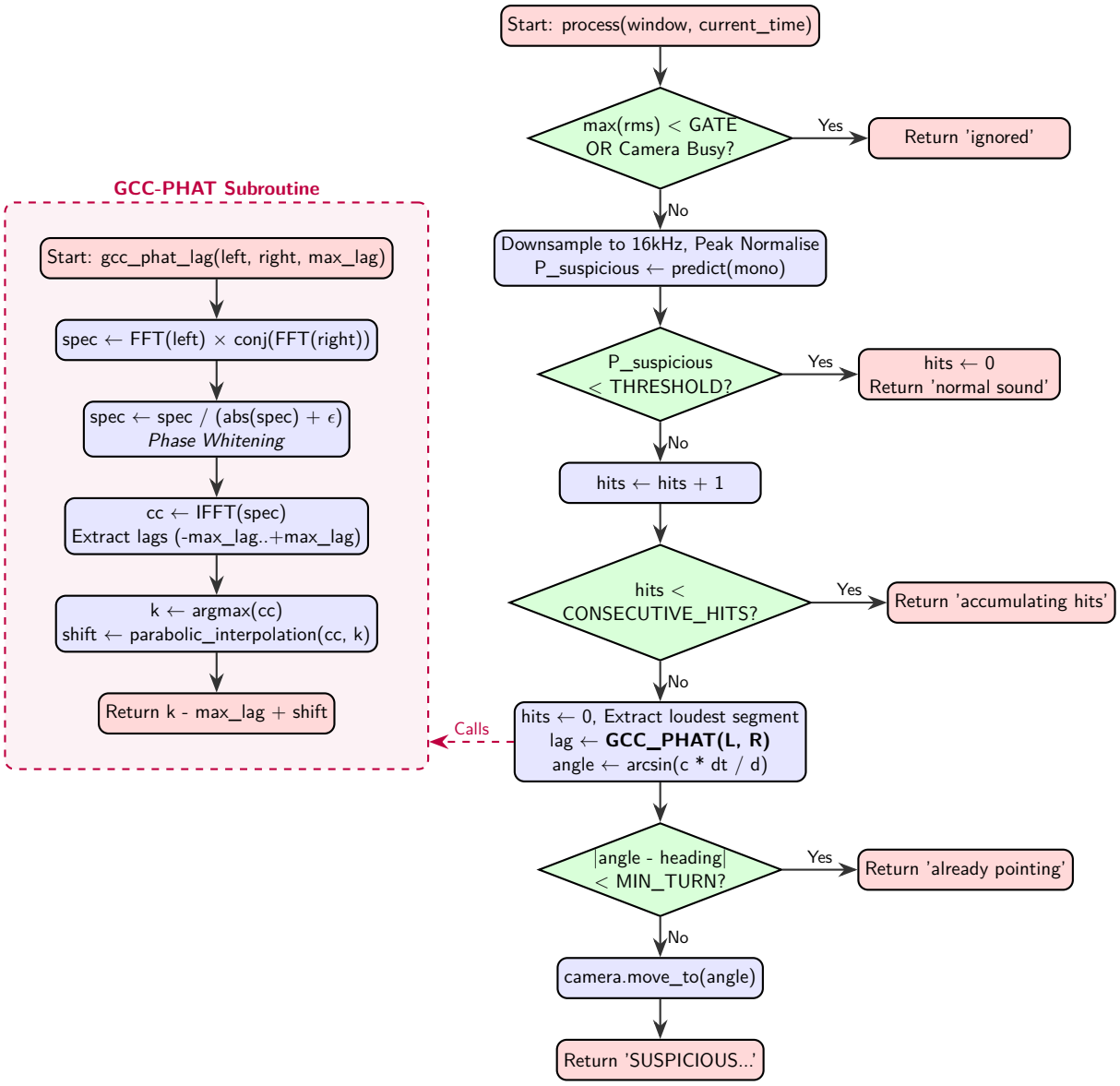


Figure 2: System Flowchart representing the pipeline steps and DOA estimation

4 Implementation, Testing and Results

4.1 Hardware Implementation

The hardware build consists of three primary components mounted rigidly together to form a single, cohesive surveillance unit. Rigid mounting is critical because the DOA calibration values (microphone distance, pan-speed measurements) are only valid for one physical configuration. If any component shifts, the system must be recalibrated.

- **Raspberry Pi:** The central processing unit runs the complete software stack, including audio capture, AI inference, DOA estimation, and camera control. The Pi's GPIO header provides the I2S bus interface (SCK on pin 12, WS on pin 35, SD on pin 38) for the two MEMS microphones.
- **Two INMP441 MEMS Microphones:** Wired to the Pi's GPIO header via a breadboard. Both microphones share the I2S bus (SCK, WS, SD), differentiated by their L/R

pin configuration: Mic 1 has L/R tied to GND (left channel, column 0). Mic 2 has L/R tied to VDD (right channel, column 1). Physical separation is 25 cm.

- **TP-Link TL-SC4171G IP Camera:** Connected to the Pi via Ethernet cable on the same subnet. The camera is controlled via HTTP GET requests to its CGI API at `/cgi-bin/operator/ptzset`. The camera address and credentials are set in `config.py`.

The microphone geometry follows the convention: column 0 (left channel) corresponds to the left microphone, column 1 (right channel) to the right microphone. A lag > 0 indicates the left channel is delayed, meaning the sound arrives from the right, and the camera should turn right. If the left and right channels appear swapped during calibration, the two L/R ties should be exchanged.

4.2 Software Implementation

The software is organised into seven Python modules. Each module has a single, well-defined responsibility.

Module	Responsibility
`main.py`	Entry point, audio loop, decision pipeline, CLI argument parsing, session statistics
`classifier.py`	TFLite model wrapper, audio preprocessing (resample, normalise), inference
`doa.py`	Direction of arrival estimation using GCC-PHAT cross-correlation
`camera.py`	PTZ camera HTTP controller with background thread, dead-reckoned heading
`config.py`	All tunable settings: model, audio, DOA, and camera parameters
`term.py`	Thread-safe, colour-coded, timestamped terminal logging
`doa_check.py`	Standalone microphone calibration tool (no model, no camera)

Key implementation details:

- **Runtime Detection:** The classifier module automatically detects and prefers lighter TFLite runtimes in the order: `ai_edge_litert`, `tflite_runtime`, `tensorflow`. This minimises memory usage on the Raspberry Pi without requiring the user to manually select a runtime.
- **Ring Buffer:** Audio blocks are accumulated in a NumPy array using `np.roll()` under a `threading.Lock`. The window length stays constant and access is simple, although `np.roll()` creates a new array on each callback; at ten callbacks per second this is an acceptable cost.
- **Background Camera Thread:** A daemon thread (`threading.Thread`) dequeues move plans from a `queue.Queue(maxsize=1)` and executes them. Only one move can be pending; while the camera is busy, new move requests are rejected rather than queued, so moves cannot pile up.
- **Continuous Move Protocol:** The command `move=<dir>&move=repeat` keeps the camera motor turning until an explicit `move=stop` is received. This enables smooth, single-move panning to any angle, replacing the old approach of sending 1.5-second chunked moves that produced jerky motion.

- **Motor Speed Calibration:** Pan speeds were empirically measured (48.5 deg/s right, 47.5 deg/s left at PanSpeed=100) by timing moves between saved preset positions at 15-degree intervals.
- **Idle Home Return:** After IDLE_HOME_SECONDS (default 20 seconds) with no move, the camera automatically returns to the home preset (0 degrees) for absolute heading re-synchronisation.

4.3 Working Model / Prototype

The system operates in three distinct modes to support different stages of development, testing, and deployment:

- **Live Mode** (`python3 src/main.py`): The primary operational mode. Stereo audio from the I2S microphones is processed in real time. When a suspicious sound is detected, the camera physically moves to face the estimated direction. The system runs continuously until terminated with Ctrl+C.
- **Dry-Run Mode** (`python3 src/main.py --dry-run`): Live audio is captured and processed through the full pipeline, but camera HTTP commands are only printed to the terminal, not actually sent. This mode is invaluable for threshold tuning and false-positive analysis without physically moving the camera.
- **File Replay Mode** (`python3 src/main.py --file recording.wav`): A WAV file is replayed through the pipeline with simulated timestamps. The camera is fully simulated (no HTTP requests, no sleep delays). WAV files are automatically resampled to 48 kHz if necessary, and mono files are duplicated to stereo (DOA will always be 0 degrees).

4.4 Testing and Validation

Microphone Calibration Testing
The microphone calibration tool (`doa_check.py`) was used to verify the physical alignment and spacing of the two INMP441 microphones. The test procedure involved playing different sounds approximately 5 times each at each of three positions (front, far left, far right) at a distance of 1-2 metres from the microphones.

Position	True Angle	Measured Angle Range	Assessment
Front (straight ahead)	0 deg	+3.3 to +3.5 deg	Small, steady offset of about +3.4 deg
Far Right	+90 deg	+70 to +78 deg	Reads 12-20 deg short of the true angle
Far Left	-90 deg	-45 to -62 deg	Reads 28-45 deg short of the true angle

Accuracy is good in front and falls towards the sides, where the arcsin conversion from time delay to angle is least well conditioned. However, the arcsin formula is symmetric, so it does not explain why the far-left error is larger than the far-right error. Reflections or a fixed channel offset are likely contributors, although this was not confirmed. The camera's field of view of approximately 53 degrees extends about 26.5 degrees either side of its heading, so a far-right source stays in frame, while a far-left source may fall outside the frame.

Model Threshold Testing
The default threshold of P(suspicious) ≥ 0.60 was selected during development testing to balance sensitivity against false activations. It is stricter than the 0.5 threshold used for the notebook

evaluation reported in Section 4.6. The threshold is exposed as a command-line argument (`--threshold`) for deployment-specific tuning. Lower thresholds increase sensitivity but also increase false positives; higher thresholds reduce false positives but risk missing genuine events.

Camera Calibration Testing

Camera pan speeds were calibrated by measuring the time required to move between saved preset positions at 15-degree intervals. This was performed with the camera's PanSpeed setting at 100 (maximum). The results were:

- Right pan speed: 48.5 deg/s
- Left pan speed: 47.5 deg/s
- Right start-up lag: 0.05 s
- Left start-up lag: 0.03 s
- Stop command: retried up to 3 times with 50 ms delay to prevent end-stop runaways

4.5 Performance Evaluation

Metric	Value
Model file size	∼3.7 MB (3,685,976 bytes, quantised TFLite)
Model inference time (Raspberry Pi)	∼7-10 ms per 1-second window
End-to-end latency (sound to camera motion)	$<$ 1 second (design target)
Processing cadence	Every 0.5 s (overlapping 1 s windows)
DOA reading, front (true 0 deg)	+3.3 to +3.5 degrees
DOA reading, far right (true +90 deg)	+70 to +78 degrees
DOA reading, far left (true -90 deg)	-45 to -62 degrees
Camera pan speed (right)	48.5 deg/s
Camera pan speed (left)	47.5 deg/s
Camera heading re-sync	Automatic via home preset at start, exit, idle, near-0 deg

4.6 Results and Discussion

The Sound Sentry system successfully demonstrates the feasibility of combining edge-AI audio classification with robotic PTZ camera control on resource-constrained hardware. When a sound is classified as suspicious, the system steers the camera towards the approximate direction of the sound source.

Key strengths of the implemented system:

- **Real-time operation:** Model inference takes only 7-10 ms of the 500 ms processing cadence, leaving headroom for the remaining pipeline stages.
- **Edge-only operation:** All processing runs on-device, so the system has no cloud latency, does not upload audio and works offline.
- **Modular architecture:** Individual components (classifier, DOA, camera controller) can be tested, calibrated, and replaced independently, facilitating iterative development.
- **Error handling:** Motor stop retries, automatic home re-sync and graceful shutdown handling allow the system to recover from common control failures.

Engineering challenges encountered and resolved during development:

#	Problem	Root Cause	Solution
1	Jerky camera movement	Camera auto-stops after 600 ms PanDelay	Used `move=<dir>&move=repeat` for continuous motion
2	Camera overshot by ∼60%	Assumed 30 deg/s; actual ∼48.5 deg/s	Calibrated against preset positions
3	Short right moves fell 1.5 deg short	Motor start-up lag underestimated at 0.03 s	Right start-up lag raised to 0.05 s; timer starts before the HTTP request
4	Camera spun to end-stop	Lost `move=stop` not retried	Retry stop 3x; home re-sync on failure
5	Heading drift over many moves	Timing errors accumulate	Periodic absolute home-preset re-sync
6	Camera stuck at wall	Wall echoes distorted DOA	Relocated hardware to open area

Quantitative Model Results

The classifier was evaluated in the training notebook on the 20% validation split (88 clips), using the dense classification head on YAMNet embeddings at a threshold of 0.5. One normal clip was misclassified as suspicious and no suspicious clips were missed. These results apply to clean dataset clips. They were not repeated for the quantised model or at the deployed threshold of 0.60, and a formal count of true and false activations in live operation is not included in this report. Live performance may differ because real environments contain sounds the model has not been trained on. For comparison, a small CNN trained on log-Mel spectrograms in the same notebook reached 76% accuracy and detected only 38% of suspicious clips, which supports the choice of transfer learning.

Metric	Value
Accuracy	99%
Precision	97%
Recall	100%
F1-Score	98%

Table 4: Classifier Performance Metrics on the Validation Set (threshold 0.5)

	Predicted Normal	Predicted Suspicious
Actual Normal	55	1
Actual Suspicious	0	32

Table 5: Confusion Matrix on the Validation Set (threshold 0.5)

4.7 Screenshots / Prototype Photographs

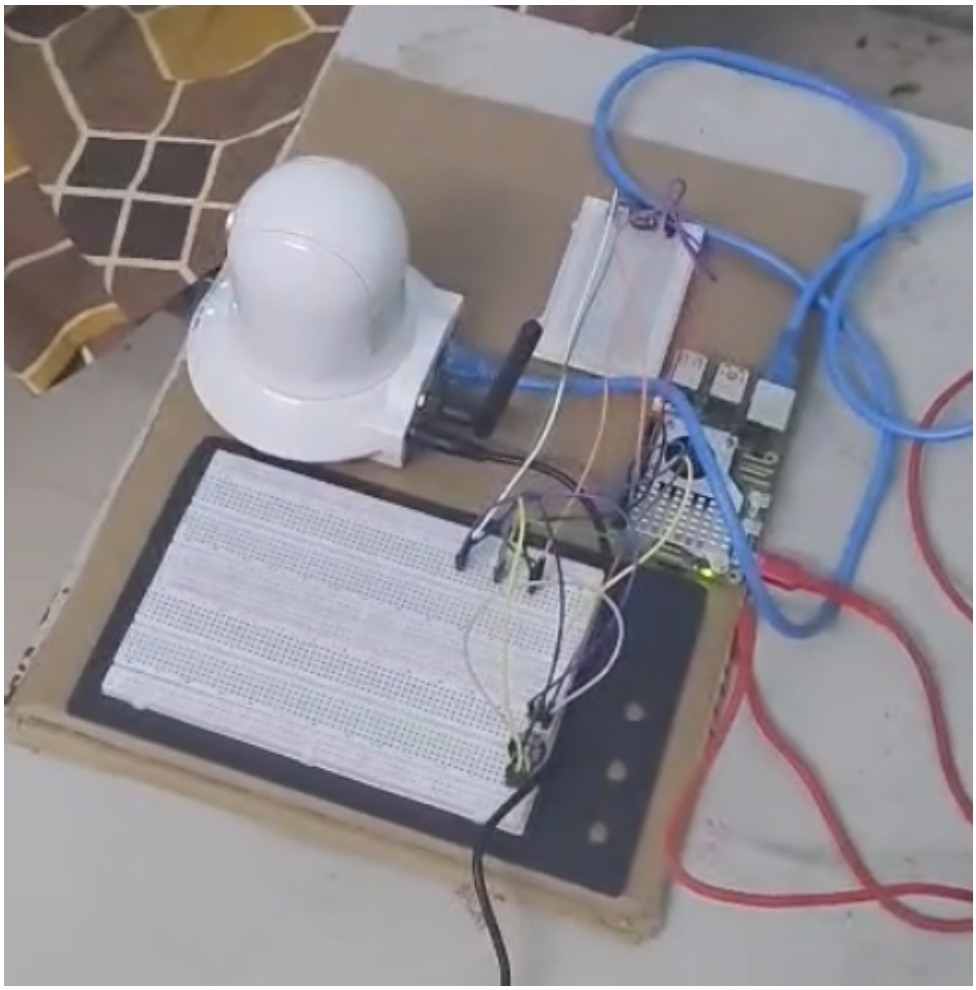


Figure 3: Hardware Prototype Setup

5 Conclusion and Future Scope

5.1 Summary of the Work

Sound Sentry is a working edge-AI security prototype that combines real-time audio classification with robotic camera control, running entirely on a Raspberry Pi without cloud services. The system continuously captures stereo audio at 48 kHz from two INMP441 I2S MEMS microphones, classifies each 1-second audio window using a quantised YAMNet-based TFLite model with approximately 7-10 ms inference time, estimates the direction of arrival using GCC-PHAT cross-correlation with sub-sample lag resolution, and steers a TP-Link TL-SC4171G pan-tilt IP camera to face the sound source.

The system demonstrates the practical feasibility of combining deep-learning-based audio intelligence with robotic actuation on resource-constrained edge hardware. The classifier achieved 99% accuracy on its ESC-50 validation set. The multi-stage decision pipeline keeps quiet audio and camera motor noise away from the model, and the background camera thread keeps audio processing running during camera moves. The main remaining weaknesses are false activations on untrained sounds and reduced direction accuracy at the far left.

5.2 Achievement of Objectives

Objective	Status	Details
Train binary audio classifier via transfer learning	Achieved	YAMNet + dense head trained on an ESC-50 subset, 99% validation accuracy, exported as quantised TFLite
Deploy model at edge with inference $\leq$ 10 ms	Achieved	$\sim$7-10 ms inference on Raspberry Pi
GCC-PHAT DOA with sub-sample resolution	Achieved	Parabolic interpolation gives fractional lag resolution; accuracy falls towards the sides
PTZ camera controller	Achieved	Dead-reckoned heading, calibrated speeds, home re-sync, retry logic
Integrated real-time pipeline	Achieved	0.5 s cadence, decision stages, background camera thread

5.3 Limitations

1. False positives on untrained sounds: The model was trained on only 11 ESC-50 classes and has limited exposure to normal but sharp sounds such as claps and door slams. Such sounds, along with speech and music, are outside its training data and may be classified as suspicious.

2. Validation on dataset clips: The classifier metrics come from ESC-50 clips rather than recordings made with the deployed microphones, so they may not reflect live performance.

3. Left/right only: Two microphones provide only a one-dimensional angle ranging from -90 to +90 degrees. The system cannot distinguish front from back and cannot determine elevation. A sound from directly behind produces the same TDOA as a sound from directly in front.

4. Reduced accuracy at the sides: The measured error reaches 12-20 degrees at far right and 28-45 degrees at far left. Since half of the camera's field of view is about 26.5 degrees, a far-left source may fall outside the frame.

5. Wall echoes: Nearby reflective surfaces (walls, floors, ceilings) produce multipath reflections that distort the TDOA estimate. The hardware should be positioned in an open area away from walls for best results.

6. Dead-reckoned heading: The camera position is estimated from motor timing rather than read back from an encoder. Small timing errors accumulate over many moves. Periodic home-preset re-synchronisation mitigates but does not eliminate this drift.

7. Single-band RMS gate: The loudness gate operates on broadband RMS energy and is not frequency-selective. A very quiet suspicious sound may fall below the gate, while a loud but diffuse background (e.g., rain on a metal roof) may pass it.

8. Deaf period during moves: Audio is ignored while the camera moves and for 2 seconds afterwards, so sounds during this period are not analysed.

9. No alert output: The current system only moves the camera. It does not send push notifications, capture snapshots, record video clips, or integrate with external alerting systems.

5.4 Future Enhancements

1. Custom Dataset: Record suspicious and non-suspicious sounds on the exact same microphone setup and acoustic environment. Retraining the model on in-situ recordings is expected to reduce false-positive rates by capturing the specific noise characteristics of the deployment environment.

2. Four-Microphone Array: Upgrading to a four-microphone array would enable 360-degree direction finding with greatly improved angular accuracy and front/back disambiguation.

3. Alert Integration: Implement push notifications via email or mobile app, including a camera snapshot attached to each detection event for immediate human review.

4. Time-of-Day Arming: Implement configurable 'armed' hours so the system only acts during specified time windows (e.g., nighttime), reducing unnecessary activations during periods of normal activity.

5. Web Dashboard: Develop a local web interface showing real-time detections, live camera feed, event history with timestamps and angles, and system statistics and health metrics.

5.5 Real-World Deployment Possibilities

With further field testing and, where needed, retraining on relevant sound classes, the system could be applied in the following settings:

- Residential Security: Low-cost, privacy-preserving perimeter monitoring for homes and apartments, operating without cloud subscriptions or internet dependency.

- Commercial Premises: Warehouse, office, and retail store surveillance with autonomous threat-direction tracking, complementing existing visual security systems.

- Outdoor and Remote Sites: Construction sites, agricultural facilities, and remote infrastructure where wired internet is unavailable or unreliable.

- Wildlife Conservation: Automated monitoring of nocturnal animal activity in nature reserves, detecting and tracking sounds of interest (poaching activity, animal distress calls).

- Smart Building Integration: Deployment as a module within a broader IoT or smart-building ecosystem, feeding detection events to building management systems via MQTT or REST APIs.

5.6 Learning Outcomes

Through this project, the team gained significant hands-on experience in the following areas:

1. Transfer Learning: Adapting a pre-trained large-scale audio model (YAMNet) to a specific binary classification task with limited training data, understanding the principles of feature reuse and domain adaptation.

2. Edge AI Deployment: Converting, quantising, and deploying TensorFlow models on resource-constrained hardware using TensorFlow Lite, including runtime selection, input tensor resizing, and performance optimisation.

3. Digital Signal Processing: Implementing GCC-PHAT cross-correlation for time-difference-of-arrival estimation, including FFT-based spectral whitening, phase-only cross-correlation, and parabolic interpolation for sub-sample lag resolution.

4. Robotics Control: Interfacing with a PTZ camera's HTTP CGI API, calibrating motor speeds against known positions, and implementing dead-reckoned heading with periodic absolute re-synchronisation.

5. Systems Integration: Combining multiple hardware components (MEMS microphones, Raspberry Pi, IP camera) and software modules into a cohesive real-time system with error handling, graceful shutdown, and comprehensive logging.

5.7 References

[1] Google, "YAMNet", TensorFlow Hub. `https://tfhub.dev/google/yamnet/1`

[2] Howard, A.G. et al., "MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications", arXiv:1704.04861, 2017.

[3] Gemmeke, J.F. et al., "Audio Set: An ontology and human-labeled dataset for audio events", ICASSP 2017.

[4] Piczak, K.J., "ESC: Dataset for Environmental Sound Classification", ACM Multimedia 2015. Kaggle copy: `https://www.kaggle.com/datasets/mmoreaux/environmental-sound-classification-50`

[5] Knapp, C.H. and Carter, G.C., "The Generalized Correlation Method for Estimation of Time Delay", IEEE Trans. Acoustics, Speech, and Signal Processing 24(4), 1976.

[6] TensorFlow Lite - `https://www.tensorflow.org/lite`

[7] AI Edge LiteRT - `https://ai.google.dev/edge/litert`

[8] TP-Link TL-SC4171G - Product documentation; CGI control via `/cgi-bin/operator/ptzset`.

[9] sounddevice - `https://python-sounddevice.readthedocs.io/`

[10] Raspberry Pi - `https://www.raspberrypi.com/`

[11] INMP441 MEMS Microphone - InvenSense (TDK) INMP441 Datasheet.

[12] Harris, C.R. et al., "Array programming with NumPy", Nature 585, 2020.

[13] Virtanen, P. et al., "SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python", Nature Methods 17, 2020.

5.8 Appendix

A. Project File Structure

```
Sound_Aware_Security_Camera/
  |-- src/
      |-- main.py
      |-- classifier.py
      |-- doa.py
      |-- camera.py
      |-- config.py
      |-- term.py
      |-- doa_check.py
  |-- models/
      |-- sentry_end_to_end_quantized.tflite
      |-- sound-model-ai-robotics.ipynb
  |-- report/
      |-- Sound_Sentry_Report.pdf
  |-- requirements.txt
  |-- README.md
  |-- .gitignore
```

B. Command-Line Interface

```
usage: main.py [-h] [--dry-run] [--file FILE] [--device DEVICE]
               [--threshold THRESHOLD] [--gate GATE] [--hits HITS]
               [--no-return] [--recenter DEG] [--list-devices]
```

Options	Description
`--dry-run`	Print camera commands, don't send them
`--file FILE`	Replay a WAV file instead of the microphone
`--device DEVICE`	Audio input device index
`--threshold THRESHOLD`	P(suspicious) threshold (default: 0.60)
`--gate GATE`	RMS loudness gate (default: 0.05)
`--hits HITS`	Consecutive suspicious windows needed (default: 1)
`--no-return`	Leave camera at final position on exit
`--recenter DEG`	Camera is at DEG: return it to 0 deg and exit
`--list-devices`	Show audio devices

C. Key Configuration Parameters

Parameter	Default	Description
MODEL_PATH	models/sentry_...tflite	Path to the TFLite model
MODEL_SAMPLE_RATE	16000	Model training sample rate (Hz)
MODEL_WINDOW_SECONDS	1.0	Audio window duration (s)
SUSPICIOUS_THRESHOLD	0.60	Minimum P(suspicious) for detection
CONSECUTIVE_HITS	1	Required consecutive windows above threshold
SAMPLE_RATE	48000	Hardware capture sample rate (Hz)
HOP_SECONDS	0.5	Classification cadence (s)
RMS_GATE	0.05	Minimum RMS to pass loudness gate
MIC_DISTANCE	0.25	Microphone spacing (m)
CAMERA_IP	172.17.137.93	Camera IP address
MOVE_MODE	continuous	Move mode (continuous / preset)
PAN_SPEED_RIGHT	48.5	Right pan speed (deg/s)
PAN_SPEED_LEFT	47.5	Left pan speed (deg/s)
COOLDOWN	2.0	Post-move audio ignore period (s)
IDLE_HOME_SECONDS	20.0	Auto-home after idle (s)